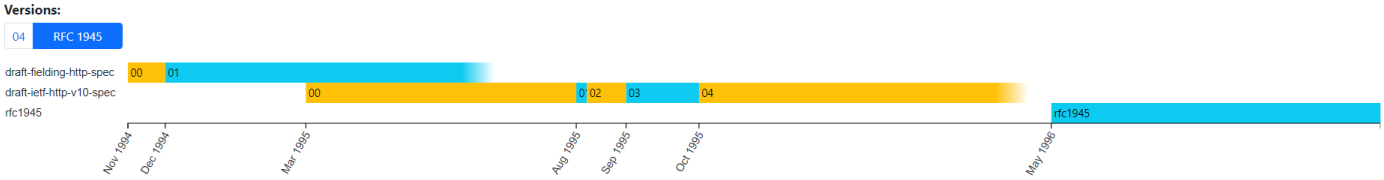
Hypertext Transfer Protocol -- HTTP/1.0

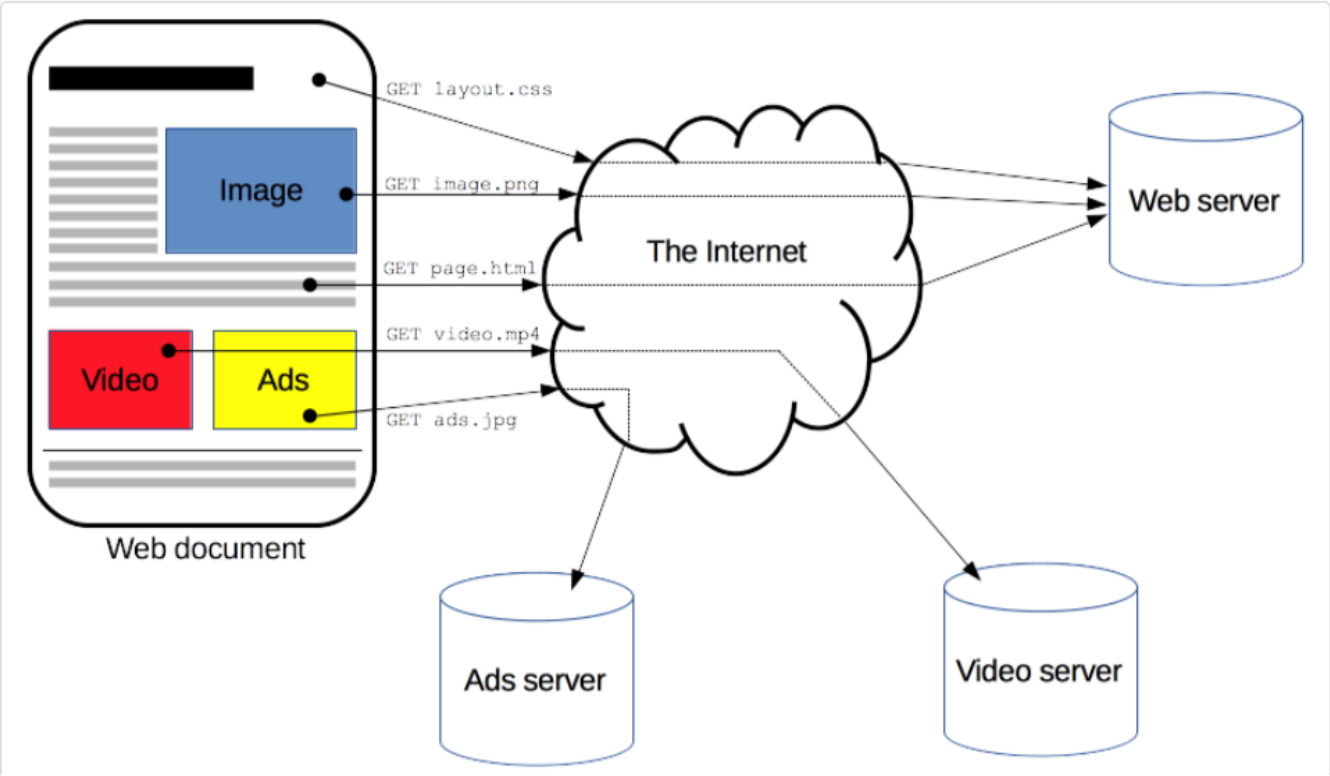
Hypertext Transfer Protocol -- HTTP/1.0

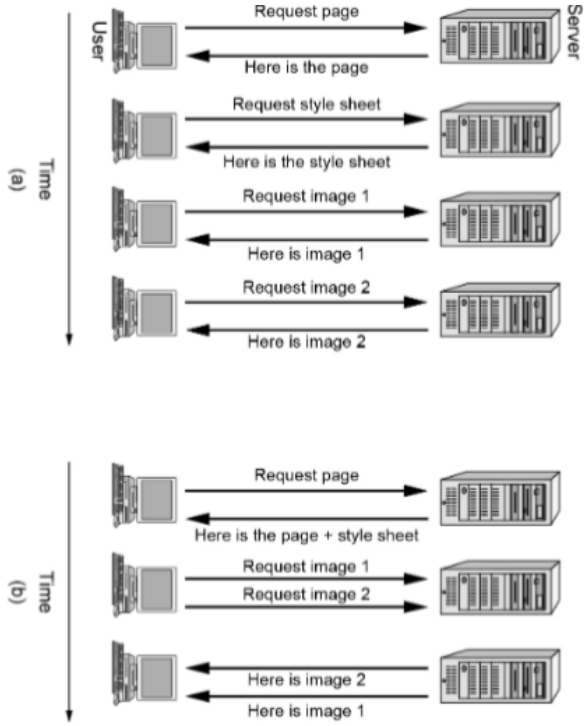
RFC 1945

Status | IESG evaluation record | IESG writeups | Email expansions | History

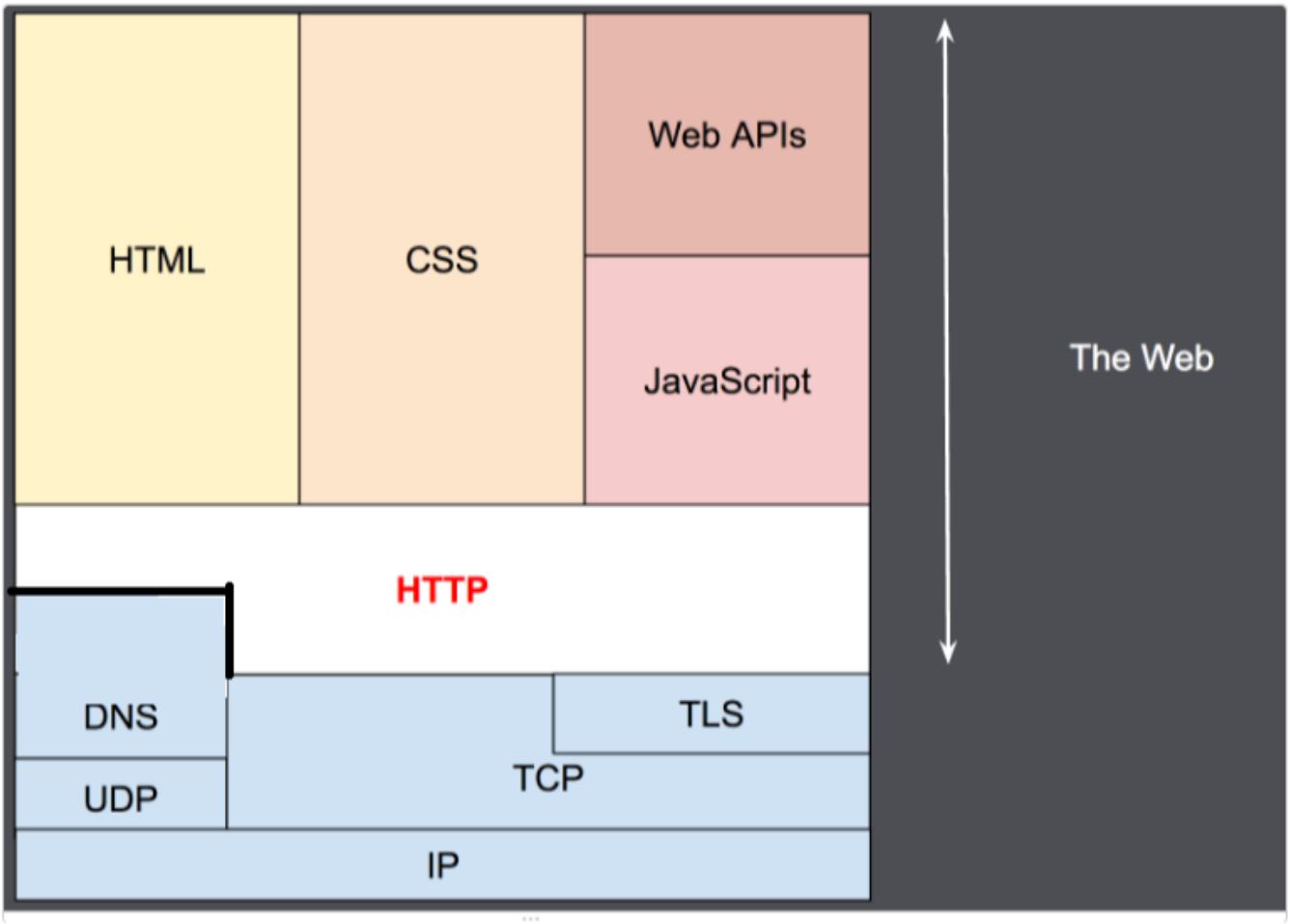


HTTP is a protocol for fetching resources such as HTML documents. It is the foundation of any data exchange on the Web and it is a client-server protocol, which means requests are initiated by the recipient, usually the Web browser. A complete document is reconstructed from the different sub-documents fetched, for instance, text, layout description, images, videos, scripts, and more.





(a) Getting a Web page in HTTP/1.1. (b) Getting the same page in HTTP/2.



A list of common features controllable with HTTP:

- ***Caching*** *(https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching)*: How documents are cached can be controlled by HTTP. The server can instruct proxies and clients about what to cache and for how long. The client can instruct intermediate cache proxies to ignore the stored document.
- *Relaxing the origin constraint*: To prevent snooping and other privacy invasions, Web browsers enforce strict separation between websites. Only pages from the **same origin** can access all the information of a Web page. Though such a constraint is a burden to the server, HTTP headers can relax this strict separation on the server side, allowing a document to become a patchwork of information sourced from different domains; there could even be security-related reasons to do so.
- *Authentication*: Some pages may be protected so that only specific users can access them. Basic authentication may be provided by HTTP, either using the `WWW-Authenticate` (https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/WWW-Authenticate) and similar headers, or by setting a specific session using **HTTP cookies** (https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies).
- ***Proxy and tunneling*** *(https://developer.mozilla.org/en-US/docs/Web/HTTP/Proxy_servers_and_tunneling)*: Servers or clients are often located on intranets and hide their true IP address from other computers. HTTP requests then go through proxies to cross this network barrier. Not all proxies are HTTP proxies. The SOCKS protocol, for example, operates at a lower level. Other protocols, like ftp, can be handled by these proxies.
- *Sessions*: Using HTTP cookies allows you to link requests with the state of the server. This creates sessions, despite basic HTTP being a state-less protocol. This is useful not only for e-commerce shopping baskets, but also for any site allowing user configuration of the output.

HTTP flow

When a client wants to communicate with a server, either the final server or an intermediate proxy, it performs the following steps:

1. Open a TCP connection: The TCP connection is used to send a request, or several, and receive an answer. The client may open a new connection, reuse an existing connection, or open several TCP connections to the servers.
2. Send an HTTP message: HTTP messages (before HTTP/2) are human-readable. With HTTP/2, these simple messages are encapsulated in frames, making them impossible to read directly, but the principle remains the same. For example:

   HTTP

   ```
   GET / HTTP/1.1
   Host: developer.mozilla.org
   Accept-Language: fr
   ```

3. Read the response sent by the server, such as:

   HTTP

   ```
   HTTP/1.1 200 OK
   Date: Sat, 09 Oct 2010 14:28:02 GMT
   Server: Apache
   Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
   ETag: "51142bc1-7449-479b075b2891b"
   Accept-Ranges: bytes
   Content-Length: 29769
   Content-Type: text/html

   <!DOCTYPE html>… (here come the 29769 bytes of the requested web page)
   ```

4. Close or reuse the connection for further requests.

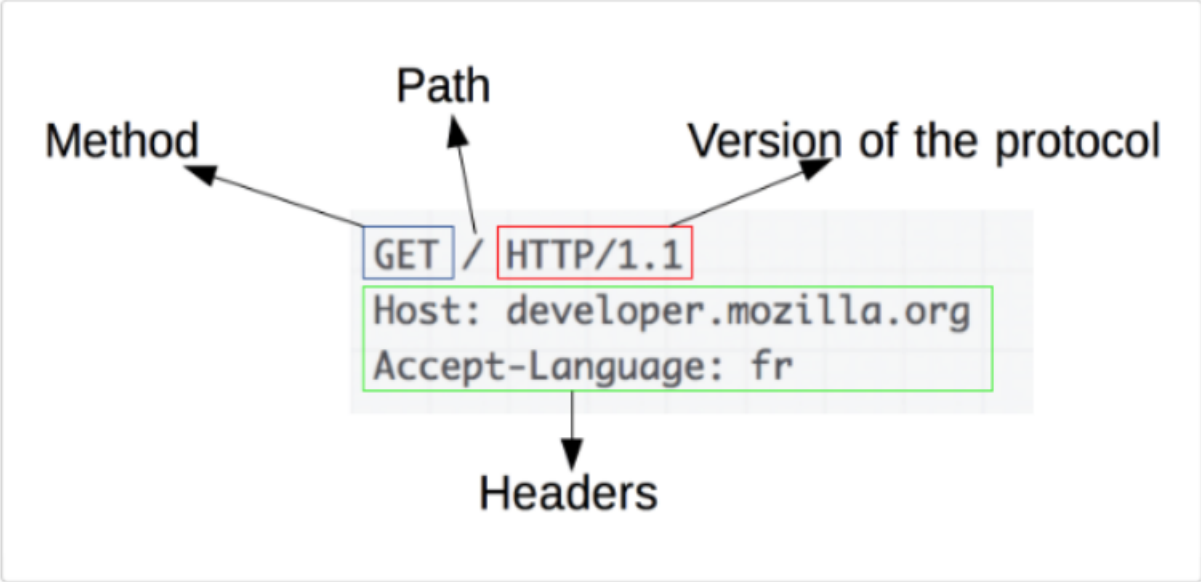
HTTP Messages

HTTP messages, as defined in HTTP/1.1 and earlier, are human-readable. In HTTP/2, these messages are embedded into a binary structure, a *frame*, allowing optimizations like compression of headers and multiplexing. Even if only part of the original HTTP message is sent in this version of HTTP, the semantics of each message is unchanged and the client reconstitutes (virtually) the original HTTP/1.1 request. It is therefore useful to comprehend HTTP/2 messages in the HTTP/1.1 format.

There are two types of HTTP messages, requests and responses, each with its own format.

Requests

An example HTTP request:



HTTP persistent connection, also called **HTTP keep-alive**, or **HTTP connection reuse**, is the idea of using a single **TCP** **(https://en.wikipedia.org/wiki/Transmission_Control_Protocol)** connection to send and receive multiple **HTTP requests** **(https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol)** /responses, as opposed to opening a new connection for every single request/response pair